eBPF を用いた低遅延温度モニタリング基盤の設計と実装

2022TC065 對馬秀悟

指導教員：宮澤元

1　はじめに

本研究では, カーネル空間での温度, 電力抑制スケジューリングを実装し, 体感を損なわない応答時間で運用可能であることを示す.

近年の OS は温度や負荷に応じた周波数制御や冷却制御を備えるが，制御の粒度や決定遅延により，応答性と温度抑制の両立が難しい場面が残る.

本研究は監視から配分決定までをカーネル内に閉じる設計により制御遅延とオーバーヘッドを抑えつつ，タスク特性に応じた段階的縮退で高温・高負荷時の安定動作を実現することを狙う.

カーネル内に観測点を設けて温度・負荷等を即時に取得し，共有データ構造に保持する．スケジューラはこの情報を基に高温・高負荷時の処理配分を滑らかに切り替える．ユーザ空間を経由しないことにより，意思決定から適用までの遅延とオーバーヘッドを最小化する．本研究の貢献は以下の点である.

- Linux 6.12 で導入された sched_ext を用いて, ユーザー空間で定義可能な温度認識型スケジューラを設計・実装した点
- タスクを優先するグループとそうでないグループに分類し, 温度上昇に伴って実行比率を動的に変化させるアルゴリズムを確立した点

2　背景

近年, エッジデバイスの高性能化が進み, 様々な場面で用いられている. しかしエッジ環境や小規模クラスタでは, 電力・冷却・空間の制約が厳しく，遅延要求も厳格であるため，エネルギー効率とサービス応答性の同時最適化が求められる.

高い周波数を維持することはそれだけ早くタスクを完了させられることでもあるが, 温度が 10 ℃上昇すると故障率が倍になるアレニウスの法則が示す様に, エッジデバイスの寿命を削る行為にもなり得る.

Linux には温度トリップポイントに応じて周波数制御や冷却デバイスを動かす Thermal　Framework ／ CPUfreq といった仕組みが用意されているが温度や使用率による判別であり, タスクの状況に応じた処理には対応しきれていない. また,MARTIN らは模倣学習と AI 処理に特化したプロセッサーを併用することで温度を最大約 17 ℃下げることに成功している [1].

しかしこれらの手法は一律な制御であることが多く, 優先したい処理も巻き込まれてしまう可能性がある.

3　設計

本研究では負荷抑制にあたりコンピュータが処理する仕事の単位であるタスクに着目する. タスクを優先したいものとそうでないものに分ける. 温度が上がるにつれて優先しないタスクの実行頻度を減らし,CPU を休ませることで温度抑制と優先したいタスクの応答性を担保する.

4　実装

システム基盤には Linux カーネルの機能を安全かつ動的に拡張するための技術である eBPF を採用した. ユーザー空間で作成したプログラムを厳格に検証した後にカーネル内に差し込むことで機能を拡張できるほか, カーネル空間での実装のため, オーバーヘッドを抑えつつ高度な機能を追加できる [2].

スケジューラには Linux6.12 で導入された sched_ext を導入した. スケジューリングポリシーを eBPF プログラムとして差し込むことができるため, 新たなポリシーの適用をカーネルの再構築なしに安全に行うことができる [3].

設計章で述べた優先度によるタスクの分配は sched_ext の機能の 1 つであるカスタム DSQ を使用した. これは実行前のタスクを溜めておくことができるキューで, ユーザー側が任意個数作成できる. 本研究では, カーネル用のタスク, 優先したいタスク, その他通常タスクの 3 つに分け, それぞれカスタム DSQ を作成した. 優先したいタスクはその PID を登録する.

また, タスクを実行するにあたって CPU のタイムスライスも変更している. 本システムでは sched_ext のマクロを使用しているが, これはデフォルトよりも長い数値となっている.

温度は ebpf のフックポイントである tracepoint/thermal/thremal_temperature で常時監視する. 温度は数値ではなく, 現在の温度を基に下 3 つの段階で管理する. 通常時の温度を COOL, 少し負荷がかかっているときを WARM, 強力な制限をかけるのが HOT とする.

5　実験

以下表 1 に実験環境を示す.

表 1　実験環境

使用機器	Raspberry Pi 5 Model(メモリ 8GB)
バージョン	Linux raspberrypi 6.12.34-v8-16k
SCX バージョン	v1.014-495-g8d646274

タスク処理にかかる時間と温度抑制を調べるため,2 つの実験を行った.

1. 同時に 100 個の負荷をかけ, 完了にかかる時間を測る
2. 長時間負荷をかけ続け, 温度がどの様に変化していくのかを測る

1 つ目の実験は 100 個の子プロセスを起動し, そのうち 1 割を優先タスクに登録し, 各子プロセスごとにかかった時間を計測した.

2 つ目の実験は 2 つの大きさが違う負荷を stress-ng コマンドで 15 分間に渡って負荷をかけ, 温度の推移を観察した.1 つは単一の大きな負荷を 15 分間, もう一方は小さめの負荷をかけ, それを繰り返し続けた. 小さめの負荷の方を優先タスクに登録した.

6 結果と考察

デフォルト時とプログラム実行時の各温度ステージの実行時間は以下の箱ひげ図のようになった.

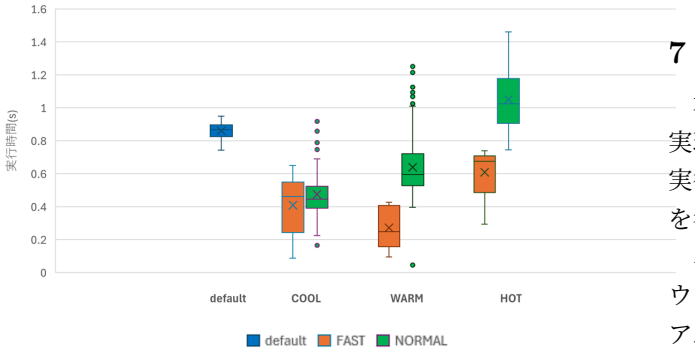

図 1 箱ひげ図

デフォルトのスケジューラは, 四分位範囲が狭く, ひげも短いため, タスクごとの実行時間のばらつきが非常に小さい. これは標準スケジューラが公平性を重視し, 全タスクに対して均等に CPU 時間を割り当てている結果である.

対して, 提案手法では, 温度ステージが上昇するにつれて, 通常タスクの実行時間が大幅に伸長し, ばらつきも増大していることがわかる. 一方で, 優先タスクの実行時間は HOT 状態であっても比較的低く抑えられている.

これは, 温度上昇に伴い通常タスクの実行権が制限され, 待機時間が増加したことによるものである. 重要なタスクの応答性能を維持するために, 通常タスクが犠牲となり, スループットが意図的に抑制されたことを示しており, 設計通りの挙動が確認できた.

温度推移のグラフを以下図 2 に, 平均, 中央値, 最高値の表を以下表 2 に示す.

実験の結果, デフォルト時と比較して平均温度で約 6.2 ℃の低下が見られた. グラフの推移を見ると, 温度上昇に伴うステージ変更により冷却制御が働き, 温度が一定範囲内で上下する推移を示していることが分かる.

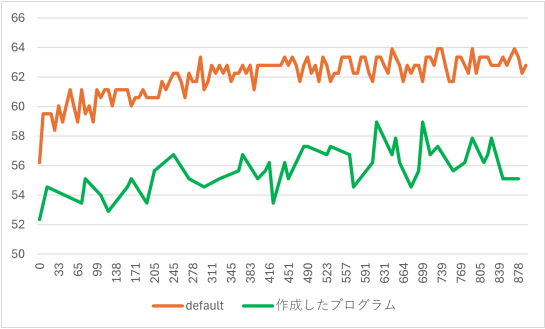

図 2 温度推移

表 2 実験結果

	デフォルト時	プログラム実行時
平均温度	62.02	55.78
中央値	62.25	55.65
最高温度	63.9	58.95

7 おわりに

本研究は優先したいタスクの応答性と温度抑制の両立を実現することを目的として高温時に優先ではないタスクの実行頻度を減らすことで CPU の負担を軽減するシステムを考案した.

ハードウェア的な冷却強化が困難な環境下でも, ソフトウェアの工夫によってシステムの信頼性と重要タスクのリアルタイム性を両立できることを示した.

今後の課題として, 優先タスクの自動判別と動的な閾値の導入が挙げられる. 優先したいタスクが登録されていない場合, 本システムでは殆どが通常タスクに振り分けられる. そのため都度 PID が変わる処理などには対応できないうえに, 図 2 のように閾値を付近を上下するような挙動になっている. また図 1 より, 登録されたタスクは迅速に処理されると分かるが, 優先タスクが空になってもシステムの挙動は変わらない. そういった場合, 挙動を変更するようにすればさらに全体の効率が上がると考えられる.

また,CPU の温度以外も対象にすることでより包括的なシステムへと発展できる可能性がある.

参考文献

[1] M.Rapp, et.al: “NPU-Accelerated Imitation Learning for Thermal Optimization of QoS-Constrained Heterogeneous Multi- Core” *2024 ACM Transactions on Design Automation of Electronic Systems, Vol.29 No1, Article 16.*

[2] eBPF “eBPF Documentation” https://ebpf.io/what-is-ebpf/ (accessed September 29.)

[3] The Linux Kernel “Extensible Scheduler Class” https://docs.kernel.org/scheduler/sched-ext.html (accessed September 29.)